

3.3 Circularly Linked Lists

Linked lists are traditionally viewed as storing a sequence of items in a linear order, from first to last. However, there are many applications in which data can be more naturally viewed as having a *cyclic order*, with well-defined neighboring relationships, but no fixed beginning or end.

For example, many multiplayer games are turn-based, with player A taking a turn, then player B, then player C, and so on, but eventually back to player A again, and player B again, with the pattern repeating. As another example, city buses and subways often run on a continuous loop, making stops in a scheduled order, but with no designated first or last stop per se. We next consider another important example of a cyclic order in the context of computer operating systems.

3.3.1 Round-Robin Scheduling

One of the most important roles of an operating system is in managing the many processes that are currently active on a computer, including the scheduling of those processes on one or more central processing units (CPUs). In order to support the responsiveness of an arbitrary number of concurrent processes, most operating systems allow processes to effectively share use of the CPUs, using some form of an algorithm known as *round-robin scheduling*. A process is given a short turn to execute, known as a *time slice*, but it is interrupted when the slice ends, even if its job is not yet complete. Each active process is given its own time slice, taking turns in a cyclic order. New processes can be added to the system, and processes that complete their work can be removed.

A round-robin scheduler could be implemented with a traditional linked list, by repeatedly performing the following steps on linked list L (see Figure 3.15):

1. process $p = L.removeFirst()$
2. Give a time slice to process p
3. $L.addLast(p)$

Unfortunately, there are drawbacks to the use of a traditional linked list for this purpose. It is unnecessarily inefficient to repeatedly throw away a node from one end of the list, only to create a new node for the same element when reinserting it, not to mention the various updates that are performed to decrement and increment the list's size and to unlink and relink nodes.

In the remainder of this section, we demonstrate how a slight modification to our singly linked list implementation can be used to provide a more efficient data structure for representing a cyclic order.

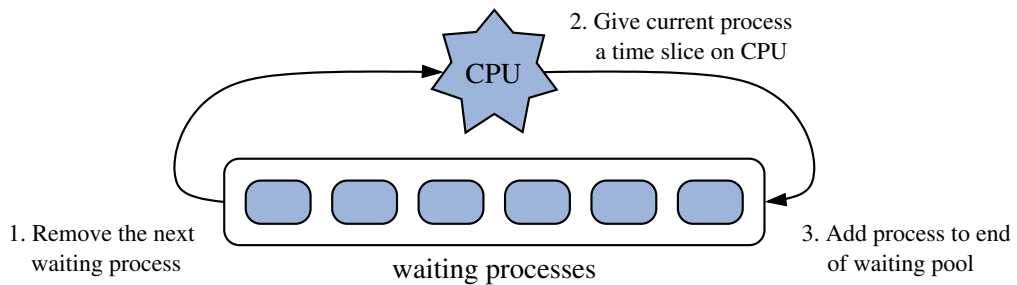


Figure 3.15: The three iterative steps for round-robin scheduling.

3.3.2 Designing and Implementing a Circularly Linked List

In this section, we design a structure known as a *circularly linked list*, which is essentially a singly linked list in which the next reference of the tail node is set to refer back to the head of the list (rather than **null**), as shown in Figure 3.16.

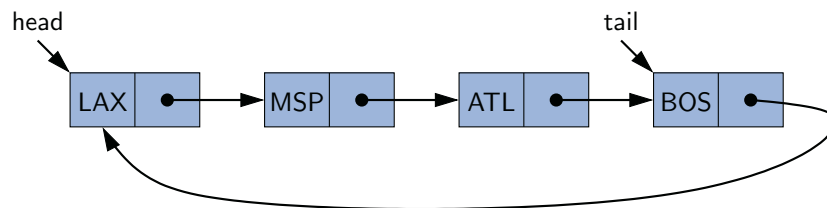


Figure 3.16: Example of a singly linked list with circular structure.

We use this model to design and implement a new `CircularlyLinkedList` class, which supports all of the public behaviors of our `SinglyLinkedList` class and one additional update method:

`rotate()`: Moves the first element to the end of the list.

With this new operation, round-robin scheduling can be efficiently implemented by repeatedly performing the following steps on a circularly linked list *C*:

1. Give a time slice to process `C.first()`
2. `C.rotate()`

Additional Optimization

In implementing a new class, we make one additional optimization—we no longer explicitly maintain the head reference. So long as we maintain a reference to the tail, we can locate the head as `tail.getNext()`. Maintaining only the tail reference not only saves a bit on memory usage, it makes the code simpler and more efficient, as it removes the need to perform additional operations to keep a head reference current. In fact, our new implementation is arguably superior to our original singly linked list implementation, even if we are not interested in the new `rotate` method.

Operations on a Circularly Linked List

Implementing the new rotate method is quite trivial. We do not move any nodes or elements, we simply advance the tail reference to point to the node that follows it (the implicit head of the list). Figure 3.17 illustrates this operation using a more symmetric visualization of a circularly linked list.

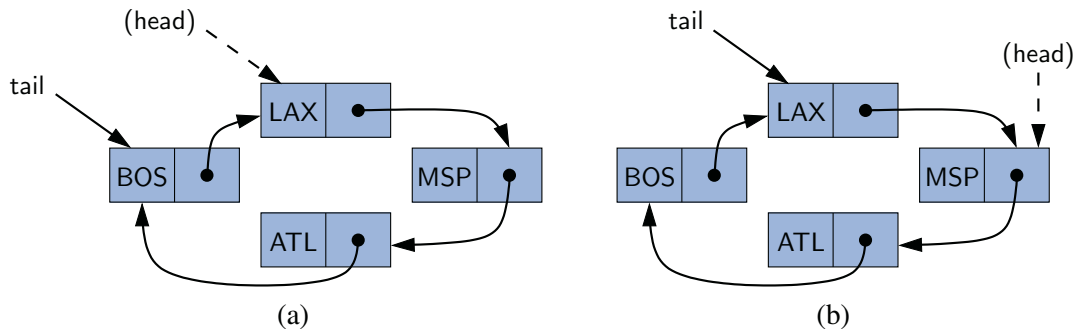


Figure 3.17: The rotation operation on a circularly linked list: (a) before the rotation, representing sequence { LAX, MSP, ATL, BOS }; (b) after the rotation, representing sequence { MSP, ATL, BOS, LAX }. We display the implicit head reference, which is identified only as `tail.getNext()` within the implementation.

We can add a new element at the front of the list by creating a new node and linking it just *after* the tail of the list, as shown in Figure 3.18. To implement the `addLast` method, we can rely on the use of a call to `addFirst` and then immediately advance the tail reference so that the newest node becomes the last.

Removing the first node from a circularly linked list can be accomplished by simply updating the next field of the tail node to bypass the implicit head. A Java implementation of all methods of the `CircularlyLinkedList` class is given in Code Fragment 3.16.

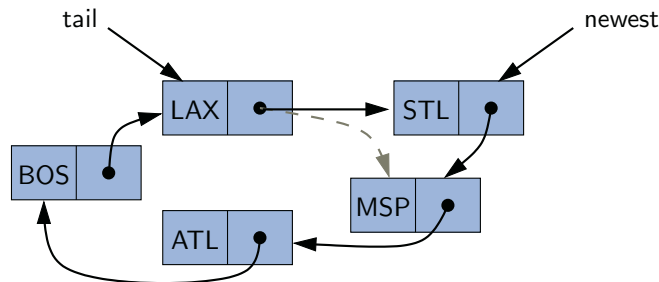


Figure 3.18: Effect of a call to `addFirst(STL)` on the circularly linked list of Figure 3.17(b). The variable `newest` has local scope during the execution of the method. Notice that when the operation is complete, STL is the first element of the list, as it is stored within the implicit head, `tail.getNext()`.

```

1  public class CircularlyLinkedList<E> {
...  (nested node class identical to that of the SinglyLinkedList class)
14  // instance variables of the CircularlyLinkedList
15  private Node<E> tail = null;           // we store tail (but not head)
16  private int size = 0;                  // number of nodes in the list
17  public CircularlyLinkedList() { }      // constructs an initially empty list
18  // access methods
19  public int size() { return size; }
20  public boolean isEmpty() { return size == 0; }
21  public E first() {                     // returns (but does not remove) the first element
22      if (isEmpty()) return null;
23      return tail.getNext().getElement(); // the head is *after* the tail
24  }
25  public E last() {                      // returns (but does not remove) the last element
26      if (isEmpty()) return null;
27      return tail.getElement();
28  }
29  // update methods
30  public void rotate() {                  // rotate the first element to the back of the list
31      if (tail != null)                  // if empty, do nothing
32          tail = tail.getNext();         // the old head becomes the new tail
33  }
34  public void addFirst(E e) {             // adds element e to the front of the list
35      if (size == 0) {
36          tail = new Node<>(e, null);
37          tail.setNext(tail);             // link to itself circularly
38      } else {
39          Node<E> newest = new Node<>(e, tail.getNext());
40          tail.setNext(newest);
41      }
42      size++;
43  }
44  public void addLast(E e) {              // adds element e to the end of the list
45      addFirst(e);                       // insert new element at front of list
46      tail = tail.getNext();             // now new element becomes the tail
47  }
48  public E removeFirst() {               // removes and returns the first element
49      if (isEmpty()) return null;         // nothing to remove
50      Node<E> head = tail.getNext();
51      if (head == tail) tail = null;      // must be the only node left
52      else tail.setNext(head.getNext()); // removes "head" from the list
53      size--;
54      return head.getElement();
55  }
56  }

```

Code Fragment 3.16: Implementation of the CircularlyLinkedList class.